

Sortarea cu lista de frecvențe

Algoritmi de sortare · Clasa a IX-a · Informatică – Python

Ce este și când o folosim?

Sortarea cu lista de frecvențe (numită și **counting sort**) este o metodă specială care **nu compară elementele între ele**. În schimb, numără de câte ori apare fiecare valoare, apoi reconstruiește lista în ordine.

Condiție obligatorie

Metoda funcționează **eficient** doar când valorile din listă sunt **numere întregi dintr-un interval limitat și cunoscut** (de exemplu, note de la 1 la 10, sau priorități de la 1 la 5).

Ideea de bază — analogie

Analogie: Cutia cu bile colorate

Ai o cutie cu bile numerotate de la 1 la 5 și vrei să le sortezi. **Nu le compari pe rând** — în schimb, iei câte un bol pentru fiecare număr și pui fiecare bilă în bolul corespunzător numărului ei. La final, torni bolurile în ordine: mai întâi bolul 1, apoi bolul 2... și gata — bilele sunt sortate!

Algoritmul — pași

- 1 Găsim valoarea **minimă** și **maximă** din listă.
- 2 Creăm o **listă de frecvențe** inițializată cu zero-uri, de lungimea intervalului de valori.
- 3 Parcurgem lista originală și **numărăm** de câte ori apare fiecare valoare (incrementăm frecvența).
- 4 **Reconstruim** lista sortată: pentru fiecare valoare posibilă, o adăugăm de atâtea ori câte apare în frecvențe.

Când NU folosim această metodă

- Când valorile nu sunt numere întregi (ex: numere reale, texte).
- Când intervalul de valori este **foarte mare** (ex: valori de la 0 la 1.000.000) — lista de frecvențe ar ocupa prea multă memorie.
- Când lucrăm cu date mixte (numere și texte în aceeași listă).

Exemplu ilustrat — note = [8, 10, 7, 9, 8, 7, 10, 8, 9, 7]

Lista inițială: note = [8, 10, 7, 9, 8, 7, 10, 8, 9, 7]

Pasul 1 - Tabelul de frecvențe construit:

Valoare	7	8	9	10
Index (val - min)	0	1	2	3
Frecvență (de câte ori apare)	3	3	2	2

Pasul 2 - Reconstrucția listei sortate:

Valoare	7	8	9	10
Frecvență	3	3	2	2
Adăugăm în lista sortată	7, 7, 7	8, 8, 8	9, 9	10, 10

Rezultat final: [7, 7, 7, 8, 8, 8, 9, 9, 10, 10] ✓

Cod Python

● COD PYTHON

```
def sortare_frecvente(lista):  
    if not lista:                                # dacă lista e goală, nu avem ce sorta  
        return lista  
  
    minim = min(lista)                          # găsim valoarea minimă  
    maxim = max(lista)                          # găsim valoarea maximă  
  
    # Creăm lista de frecvențe – câte zero-uri câte valori distincte avem  
    frecventa = [0] * (maxim - minim + 1)  
  
    # Numărăm de câte ori apare fiecare valoare  
    for x in lista:  
        frecventa[x - minim] += 1  
  
    # Reconstruim lista sortată  
    lista_sortata = []
```

```

for i in range(len(frecventa)):
    lista_sortata += [i + minim] * frecventa[i] # adăugăm valoarea de frecventa[i] ori

return lista_sortata

note = [8, 10, 7, 9, 8, 7, 10, 8, 9, 7]
print("Note originale:", note)
print("Note sortate: ", sortare_frecvente(note))
# Rezultat: Note sortate: [7, 7, 7, 8, 8, 8, 9, 9, 10, 10]

```

Caracteristici ale metodei

Proprietate	Valoare / Detalii
Complexitate timp	$O(n + k)$ — unde k este mărimea intervalului de valori
Complexitate spațiu	$O(k)$ — avem nevoie de memorie extra pentru frecvențe
Comparări	Zero — elementele nu sunt comparate niciodată!
Date potrivite	Liste mari cu valori întregi într-un interval mic și cunoscut
Dezavantaj	Nu funcționează cu date non-întregi sau intervale mari

Exerciții

Ex. 1 Nivel: Înțelegere + Implementare

Implementează algoritmul de sortare cu lista de frecvențe pentru ordonarea **notelor la testul de informatică** al unei clase de elevi (note întregi între 1 și 10).

- Citește de la tastatură numărul de elevi, apoi notele lor.
- Construiește lista de frecvențe numărând frecvența fiecărei note.
- Afișează notele sortate prin parcurgerea listei de frecvențe.

Ex. 2 Nivel: Analiză + Argumentare

Sistemul de **triaj al pacienților** dintr-un spital ordonează pacienții în funcție de gravitatea stării lor medicale, codificată cu valori de la 1 la 5 (5 = stare critică, necesită atenție imediată).

- Argumentează de ce sortarea cu lista de frecvențe este **metoda potrivită** pentru acest sistem, ținând cont de caracteristicile datelor.
- Implementează programul care afișează pacienții în ordinea **descrescătoare** a gravității (cei mai gravi, primii).

Ex. 3 Nivel: Sintează

O companie de lifturi primește zilnic cereri de mentenanță clasificate după **tipul defecțiunii** (codificat cu valori de la 1 la 8).

Implementează un program care:

1. Citește numărul de cereri și tipul defecțiunii pentru fiecare cerere.
2. Sortează cererile crescător după tipul defecțiunii folosind lista de frecvențe.
3. Afișează lista cererilor sortată și numărul de cereri pentru fiecare tip de defecțiune.